

PROJEKT

STEROWNIKI ROBOTÓW

Założenia projektowe

Robot podążający za linią

Line Follower

Skład grupy:

Mateusz Duda, 278331

Łukasz Lysek, 278341

Termin: wtTN15

Prowadzący:

dr inż. Wojciech DOMSKI

1 lutego 2026

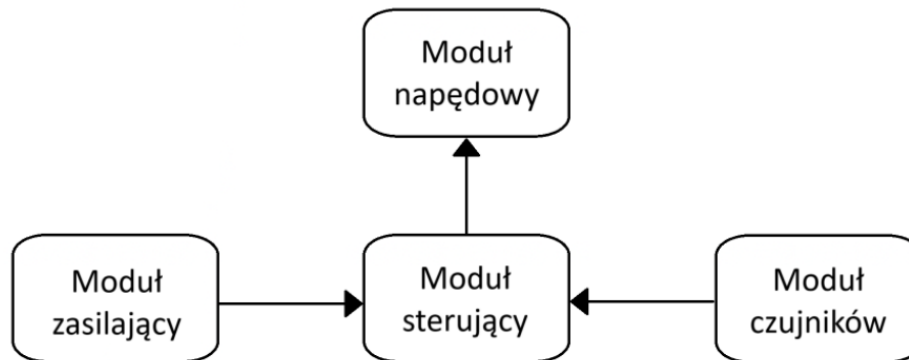
Spis treści

1	Opis projektu	2
1.1	Założenia projektowe	2
2	Konfiguracja mikrokontrolera	2
2.1	Konfiguracja pinów	5
2.2	USART	5
2.3	ADC	6
2.4	DMA	8
2.5	TIM	8
3	Urządzenia zewnętrzne	9
3.1	Czujnik linii QTR-8A	9
3.2	Przetwornica step-down AP63357	9
3.3	Sterownik silników TB6612FNG	9
3.4	Silniki szczotkowe N20-BT43	10
4	Schemat elektryczny	10
5	Sterowanie P	11
6	Sterowanie PID	12
6.1	Kod programu	12
6.2	Opis pętli sterującej	16
6.2.1	Odczyt danych i wyznaczenie uchybu	17
6.2.2	Algorytm wykrywania zgubienia linii i pokonywania ostrych zakrętów	17
6.3	Dobór nastaw regulatora PID	17
7	Trasa robota	19
8	Konstrukcja mechaniczna	19
8.0.1	Podwozie robota	19
8.1	Pojemnik na koszyk z bateriami	21
8.2	Mocowania silników	22
8.3	Mocowania listwy z czujnikami	23
8.4	Podkładka po koło podporowe	24
8.5	Moduł napędowy	25
8.6	Złożenie robota	26
9	Harmonogram pracy	27
9.1	Zakres prac	27
9.2	Podział pracy	28
10	Podsumowanie	28
	Bibliografia	29

1 Opis projektu

Celem projektu jest zaprojektowanie i zbudowanie robota podążającego za linią z wykorzystaniem płytki STM32 NUCLEO-L476RG. Robot będzie posiadał dwa koła napędzane silnikami DC, jedno koło podporowe oraz czujnik linii. Za zasilanie odpowiadać będą 4 baterie typu AA wraz z przetwornicą step-down niezbędną do poprawnego działania układów. Konstrukcja zostanie wykonana na podwoziu wydrukowanym w drukarce 3D, zaś do montażu poszczególnych elementów zostaną wykorzystane taśmy montażowe oraz śruby.

Na poniższym rysunku zamieszczono diagram reprezentujący architekturę systemu:



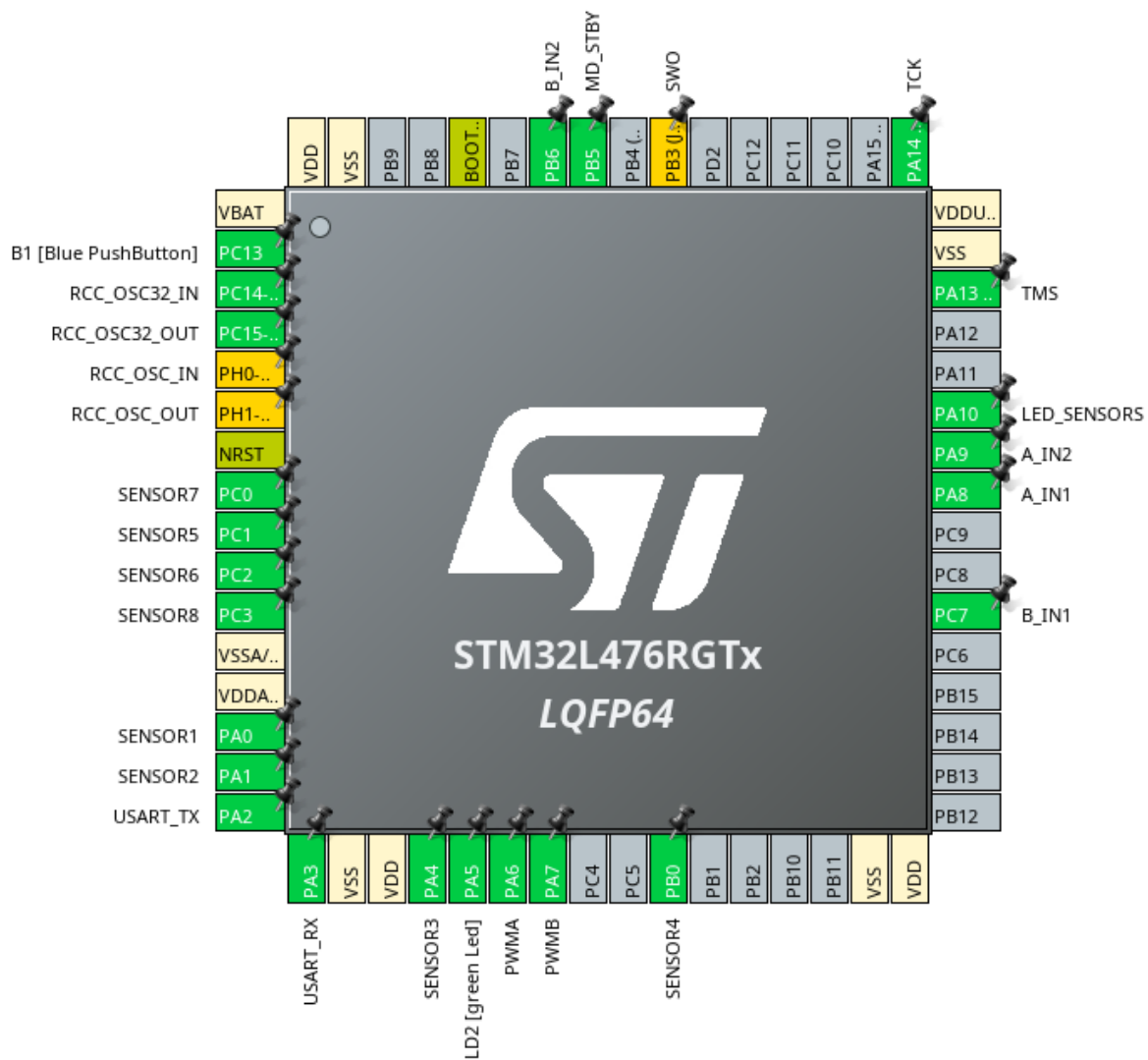
Rysunek 1: Architektura systemu

1.1 Założenia projektowe

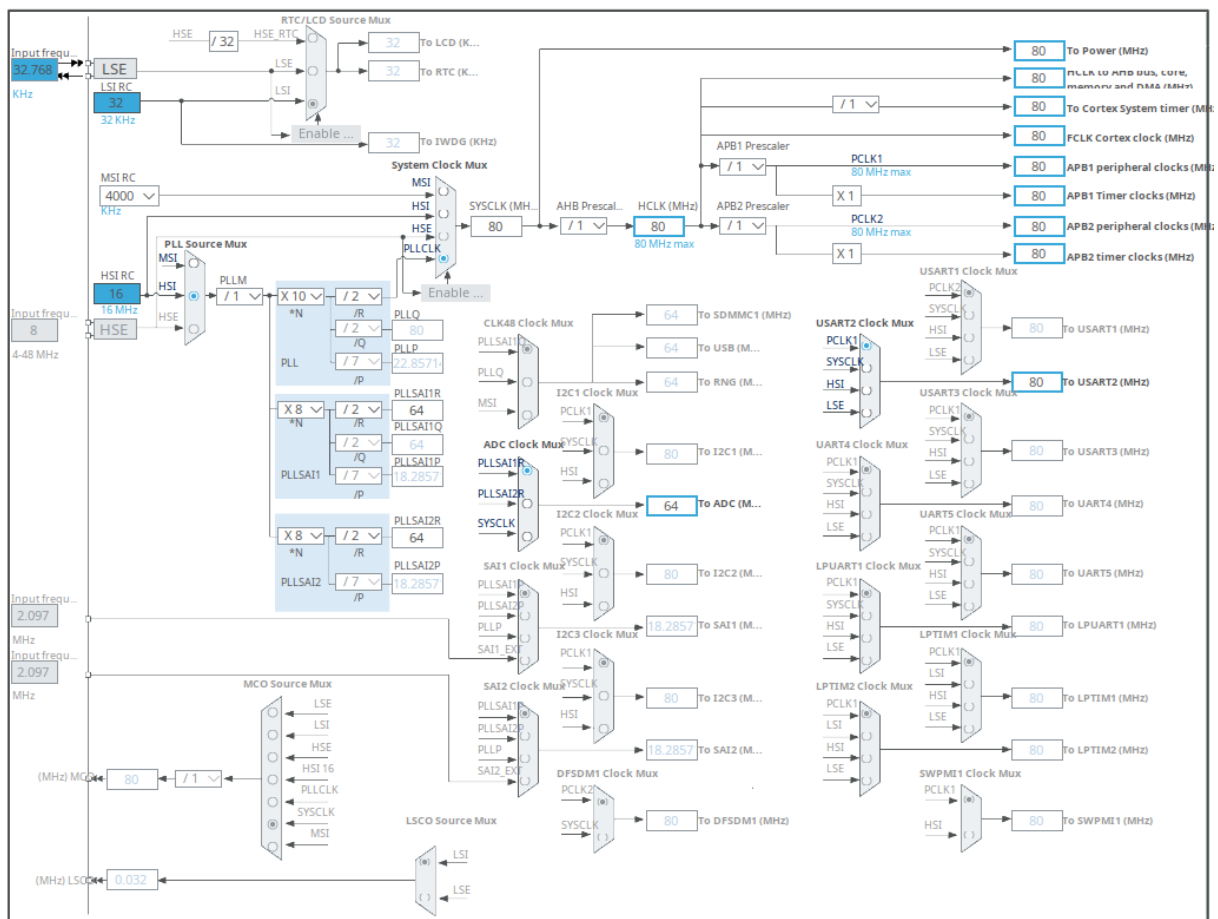
- Robot będzie wyposażony w mikrokontroler sterujący STM32 NUCLEO-L476RG.
- Za napęd odpowiadać będą dwa silniki N20-BT43 oraz sterownik Pololu 713 TB6612FNG.
- Do przemieszczania się wykorzystane zostaną dwa koła napędzane Pololu 1420 oraz jedno koło obrotowo podporowe Pololu 953.
- Robot będzie wykrywał śledzoną linię przy pomocy listwy z czujnikami odbiciowymi QTR-8A Pololu 960.
- Za zasilanie robota odpowiedzialne będą ogniwa li-ion umieszczone w specjalnym koszyku oraz przetwornica step-down 5V 3A AP63357 SparkFun COM-21256.
- Konstrukcja robota zostanie oparta na podwoziu wydrukowanym w technologii 3D, poszczególne elementy zostaną zamocowane przy pomocy śrub oraz taśm montażowych.
- Zaimplementowane zostaną algorytmy śledzenia linii i korekcji toru ruchu.

2 Konfiguracja mikrokontrolera

Ta część dokumentu opisuje konfigurację peryferiów mikrokontrolera. Przedstawia ona konfigurację wyjść mikrokontrolera, zegarów mikrokontrolera, interfejsu szeregowego USART, przetwornika analogowo-cyfrowego ADC, zarządzania bezpośrednim dostępem do pamięci DMA oraz licznika TIM.



Rysunek 2: Konfiguracja wyjść mikrokontrolera w programie STM32CubeIDE



2.1 Konfiguracja pinów

Numer pinu	PIN	Tryb pracy	Funkcja/etykieta
2	PC13	ANTI_TAMP GPIO_EXTI13	B1 [Blue PushButton]
3	PC14	OSC32_IN* RCC_OSC32_IN	
4	PC15	OSC32_OUT* RCC_OSC32_OUT	
5	PH0	OSC_IN* RCC_OSC_IN	RCC_OSC_OUT
6	PH1	OSC_OUT*	
8	PC0	ADC1_IN1	
9	PC1	ADC1_IN2	SENSOR5
10	PC2	ADC1_IN3	SENSOR6
11	PC3	ADC1_IN4	SENSOR8
14	PA0	ADC1_IN5	SENSOR1
15	PA1	ADC1_IN6	SENSOR2
16	PA2	USART2_TX	USART_TX
17	PA3	USART2_RX	USART_RX
20	PA4	ADC1_IN9	SENSOR3
21	PA5	GPIO_Output	LD2 [Green Led]
22	PA6	TIM3_CH1	PWMA
23	PA7	TIM3_CH2	PWMB
26	PB0	ADC1_IN15	SENSOR4
38	PC7	GPIO_Output	B_IN1
41	PA8	GPIO_Output	A_IN1
42	PA9	GPIO_Output	A_IN2
43	PA10	GPIO_Output	LED_SENSORS
46	PA13*	SYS_JTMS-SWDIO	TMS
49	PA14*	SYS_JTCK-SWCLK	TCK
55	PB3*	SYS_JTDO-SWO	SWO
57	PB5	GPIO_Output	MD_STBY
58	PB6	GPIO_Output	B_IN2

Tabela 1: Konfiguracja pinów mikrokontrolera

2.2 USART

Konfiguracja interfejsu szeregowego USART2 została przeprowadzona w oparciu o [2]. Interfejs ten jest wykorzystywany w trybie asynchronicznym. Jego zadaniem jest ułatwienie debugowania.

Parametr	Wartość
Baud Rate	11520
Word Length	8 Bits (including parity)
Parity	None
Stop Bits	1
Data Direction	Receive and Transmit
Over Sampling	16 Samples
Single Sample	Disable
Auto Baudrate	Disable
TX Pin Active Level Inversion	Disable
RX Pin Active Level Inversion	Disable
Data Inversion	Disable
TX and RX Pins Swapping	Disable
Overrun	Enable
DMA on RX Error	Enable
MSB First	Disable

Tabela 2: Konfiguracja peryferium USART2

2.3 ADC

Konfiguracja przetwornika analogowo-cyfrowego ADC1 została przeprowadzona w oparciu o [1]. Wykorzystywane zostało 8 kanałów przetwornika ADC1. Każdy z kanałów odpowiedzialny jest za jeden z czujników znajdujących się na listwie. Poprawne skonfigurowanie przetwornika jest niezbędne do realizacji algorytmu wykrywania linii.

Kanał	Tryb
IN1	IN1 Single-ended
IN2	IN2 Single-ended
IN3	IN3 Single-ended
IN4	IN4 Single-ended
IN5	IN5 Single-ended
IN6	IN6 Single-ended
IN9	IN9 Single-ended
IN15	IN15 Single-ended

Tabela 3: Konfiguracja kanałów peryferium ADC1

Parametr	Wartość
Mode	Independent mode
Clock Prescaler	Asynchronous clock mode divided by 10
Resolution	ADC 12-bit resolution
Data Alignment	Right alignment
Scan Conversion Mode	Enabled
Continous Conversion Mode	Enabled
Discontinous Conversion Mode	Enabled
End Of Conversion Selection	End of single conversion
Overrun behaviour	Overrun data preserved
Low Power Auto Wait	Disabled
Enable Regular Conversions	Enable
Enable Regular Oversampling	Disabled
Number Of Conversion	8
External Trigger Conversion Source	Regular Conversion launched by software
External Trigger Conversion Edge	None
Rank	1
Channel	Channel 5
Sampling Time	640.5 Cycles
Offset Number	No offset
Rank	2
Channel	Channel 6
Sampling Time	640.5 Cycles
Offset Number	No offset
Rank	3
Channel	Channel 5
Sampling Time	640.5 Cycles
Offset Number	No offset
Rank	4
Channel	Channel 15
Sampling Time	640.5 Cycles
Offset Number	No offset
Rank	5
Channel	Channel 2
Sampling Time	640.5 Cycles
Offset Number	No offset
Rank	6
Channel	Channel 3
Sampling Time	640.5 Cycles
Offset Number	No offset
Rank	7
Channel	Channel 1
Sampling Time	640.5 Cycles
Offset Number	No offset
Rank	8
Channel	Channel 4
Sampling Time	640.5 Cycles
Offset Number	No offset
Enable Injected Conversions	Disable
Enable Analog WatchDog1 Mode	false
Enable Analog WatchDog2 Mode	false
Enable Analog WatchDog3 Mode	false

Tabela 4: Konfiguracja peryferium ADC1

2.4 DMA

Konfiguracja bezpośredniego zarządzania pamięcią DMA1 została zrealizowana w oparciu o [1] oraz [5]. DMA jest wykorzystywane w projekcie w celu zaoszczędzenia czasu jednostce operacyjnej, gdyż jego zastosowanie sprawia, że CPU jest omijane w procesie odczytywania danych z rejestru ADC.

Parametr	Wartość
DMA request	ADC1
Stream	DMA1_Channel1
Direction	Peripheral To Memory
Priority	Low

Tabela 5: Konfiguracja peryferium DMA

Parametr	Wartość
Mode	Circular
Peripheral Increment	Disable
Memory Increment	Enable
Peripheral Data Width	Half Word
Memory Data Width	Half Word

Tabela 6: Konfiguracja peryferium DMA1

2.5 TIM

Konfiguracja licznika TIM3 została zrealizowana w oparciu o [3]. W projekcie wykorzystywane są dwa kanały TIM3. Pierwszy z nich odpowiada za generację sygnału PWM dla jednego z silników, a drugi dla drugiego. Zdefiniowano dwie zmienne pomocnicze TIM3_PRESCALER = 3 oraz TIM3_PERIOD = 999.

Parametr	Wartość
Clock Source	Internal Clock
Channel1	PWM Generation CH1
Channel2	PWM Generation CH2
Prescaler (PSC - 16 bits value)	TIM3_PRESCALER
Counter Mode	Up
Counter Period (AutoReload Register - 16 bits value)	TIM3_PERIOD
Internal Clock Division (CKD)	No Division
auto-reload preload	Disable
Master/Slave Mode (MSM bit)	Disable (Trigger input effect not delayed)
Trigger Event Selection TRGO	Reset (UG bit from TIMx_EGR)
Clear Input Source	Disable
Mode	PWM mode 1
Pulse (16 bits value)	Up
Output compare preload	Enable
Fast Mode	Disable
CH Polarity	High
Mode	PWM mode 1
Pulse (16 bits value)	Up
Output compare preload	Enable
Fast Mode	Disable
CH Polarity	High

Tabela 7: Konfiguracja peryferium TIM3

3 Urządzenia zewnętrzne

3.1 Czujnik linii QTR-8A

Moduł czujnika linii QTR-8A wyposażony jest w szereg wyprowadzeń umożliwiających zasilanie, sterowanie diodami IR oraz odczyt sygnałów analogowych. Poniżej przedstawiono szczegółowy opis poszczególnych pinów:

- **VCC** – wejście napięcia zasilania modułu. Układ zasilany jest napięciem 3.3V, ponieważ przetworniki ADC w mikrokontrolerach stm32 są przystosowane do pracy w przedziale [0; 3.3] V.
- **GND** – masa układu (Ground). Musi być ona połączona ze wspólną masą całego systemu (mikrokontrolera oraz zasilania silników), aby zapewnić poprawny punkt odniesienia dla pomiarów napięcia.
- **SENSOR1** – **SENSOR8** – osiem niezależnych wyjść analogowych odpowiadających poszczególnym parom dioda-fototranzystor.
 - Każde wyjście podaje napięcie proporcjonalne do ilości światła podczerwonego odbitego od podłoża.
 - **Niskie napięcie (bliskie 0 V)**: oznacza silne odbicie światła (jasne tło/powierzchnia biała), co powoduje nasycenie fototranzystora.
 - **Wysokie napięcie (bliskie VCC)**: oznacza słabe odbicie lub brak odbicia (ciemna linia/powierzchnia czarna), co powoduje zatkanie fototranzystora.

3.2 Przetwornica step-down AP63357

Przetwornica step-down jest wykorzystywana w celu obniżenia napięcia 7.4V wychodzącego z koszyka na baterie na 5V. Jest to niezbędne w projekcie, gdyż mikrokontroler sterujący NUCLEO-L476RG wymaga napięcia zasilania równego 5V. Poniżej przedstawiono szczegółowy opis poszczególnych pinów:

- **VIN** – wejście napięcia pochodzącego z koszyka na baterie.
- **GND** – masa układu. Musi być ona połączona ze wspólną masą całego systemu, aby zapewnić poprawny punkt odniesienia dla pomiarów napięcia.
- **5V** – wyjście przetwornicy. Na wyjściu powinno pojawiać się napięcie bliskie 5V, które wykorzystywane jest do zasilania mikrokontrolera sterującego.

3.3 Sterownik silników TB6612FNG

Do sterowania elementami wykonawczymi wykorzystano dwukanałowy moduł oparty na układzie scalonym TB6612FNG. Jest to podwójny mostek H, który umożliwia niezależną kontrolę prędkości i kierunku obrotów dwóch silników prądu stałego przy użyciu sygnałów logicznych z mikrokontrolera.

Opis wyprowadzeń układu:

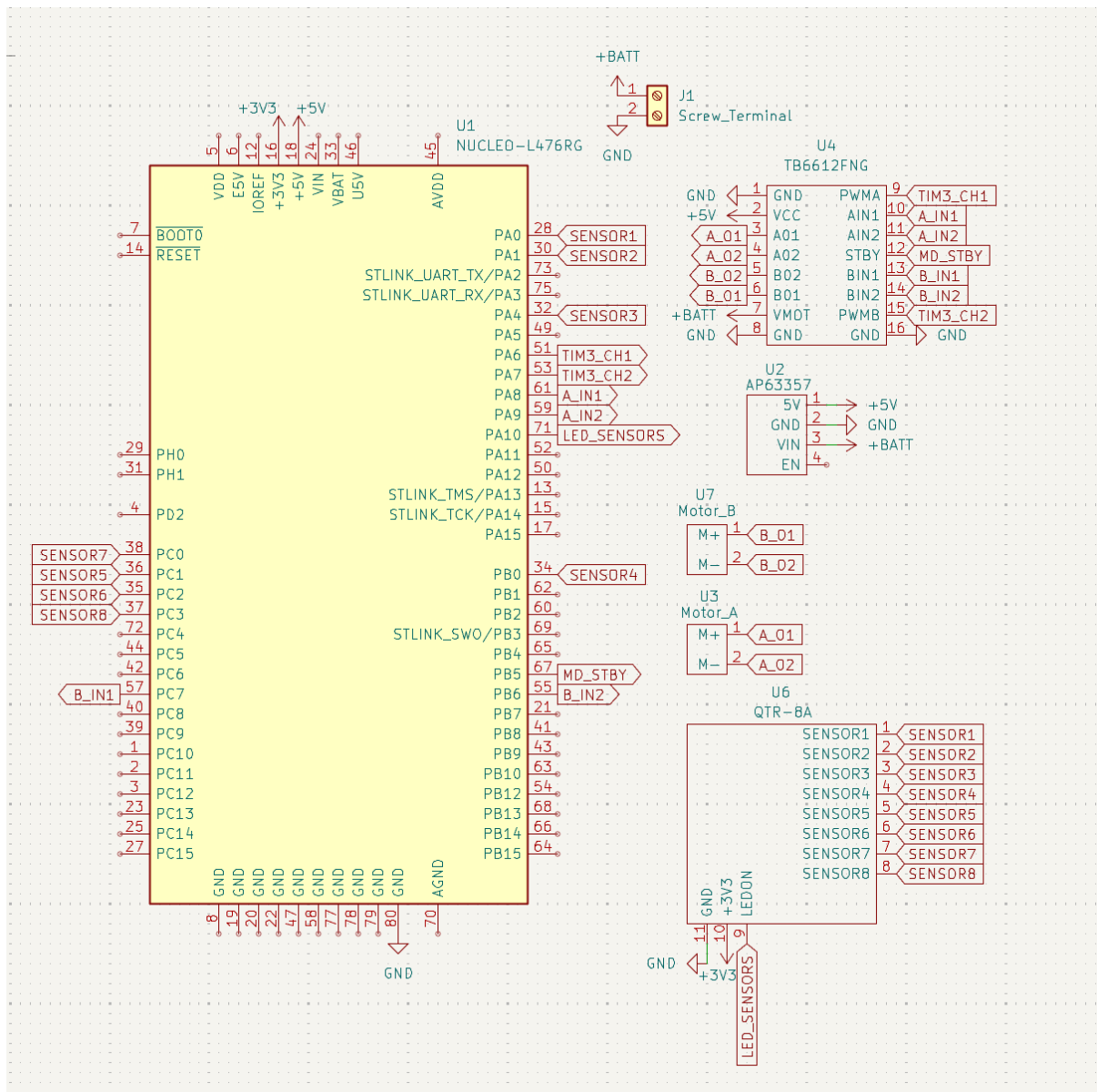
- **VMOT** – wejście napięcia zasilania silników (bezpośrednio z akumulatora).
- **VCC** – wejście zasilania logiki sterownika (5 V).
- **GND** – wspólna masa układu zasilania i sterowania.
- **PWMA** / **PWMB** – wejścia sygnału PWM służące do regulacji prędkości obrotowej (odpowiednio dla kanału A i B).
- **AIN1**, **AIN2** / **BIN1**, **BIN2** – wejścia cyfrowe ustalające kierunek obrotów silników (lewo, prawo, hamowanie).
- **STBY** – pin aktywacji układu (Standby). Wymaga podania stanu wysokiego, aby sterownik pracował; stan niski wyłącza silniki.
- **A01**, **A02** / **B01**, **B02** – wyjścia prądowe do podłączenia zacisków silników.

3.4 Silniki szczotkowe N20-BT43

Jako elementy napędzające robota zastosowano dwa miniaturowe silniki prądu stałego (DC) o napięciu znamieniowym 6V. Jeden z nich odpowiada za napędzanie prawego koła, a drugi lewego. Prędkość każdego z silników regulowana jest niezależnie za pomocą osobnych sygnałów PWM.

4 Schemat elektryczny

W oparciu o zrealizowaną konfigurację pinów oraz zakładane połączenia wyprowadzeń z koszyka na baterie wykonano schemat elektryczny przedstawiający wszystkie wyjścia, wejścia oraz relacje między pinami.



Rysunek 4: Schemat układu

5 Sterowanie P

```
1 uint16_t adc_results[8];
2 int ADC_flag;
3 char msg[256];
4
5 const int weights[8] = {40, 30, 20, 10, -10, -20, -30, -40};
6
7 float position_error = 0.0f;
8 int32_t numerator = 0;
9 int32_t denominator = 0;
10
11 float Kp = 5.0f;
12 int base_speed = 500;
13 int max_speed = 799;
14
15 int left_motor_pwm = 0;
16 int right_motor_pwm = 0;
```

Listing 1: Zmienne globalne

```
1 // 1. Włączenie czujnika linii
2 HAL_GPIO_WritePin(LED_SENSORS_GPIO_Port, LED_SENSORS_Pin, GPIO_PIN_SET);
3 HAL_Delay(20);
4
5 // 2. Start pomiarów z czujnika
6 HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_results, 8);
7
8 // 3. Ustawienie kierunku silników
9 HAL_GPIO_WritePin(A_IN1_GPIO_Port, A_IN1_Pin, GPIO_PIN_RESET);
10 HAL_GPIO_WritePin(A_IN2_GPIO_Port, A_IN2_Pin, GPIO_PIN_SET);
11 HAL_GPIO_WritePin(B_IN1_GPIO_Port, B_IN1_Pin, GPIO_PIN_RESET);
12 HAL_GPIO_WritePin(B_IN2_GPIO_Port, B_IN2_Pin, GPIO_PIN_SET);
13
14 // 4. Włączenie silników
15 HAL_GPIO_WritePin(MD_STBY_GPIO_Port, MD_STBY_Pin, GPIO_PIN_SET);
16
17 // 5. PWM dla silników
18 __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 0);
19 __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 0);
20 HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
21
22 // 6. Naciśnięcie przycisku odpala robota
23 while (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_SET)
24 {
25     HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
26     HAL_Delay(100);
27 }
28
29 HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);
30 HAL_Delay(1000);
31
32 HAL_TIM_Base_Start(&htim6);
```

Listing 2: Inicjalizacja peryferiów

```
1 while (1)
2 {
3     if (ADC_flag == 1) {
4         ADC_flag = 0;
5
6         numerator = 0;
7         denominator = 0;
8     }
```

```

9     for (int i = 0; i < 8; i++) {
10         int val = adc_results[i];
11
12         if (val < 500) val = 0;
13         numerator += val * weights[i];
14         denominator += val;
15     }
16
17     if (denominator > 100) {
18         position_error = (float)numerator / (float)denominator;
19     }
20
21     int correction = (int)(position_error *Kp);
22
23     int right_target = base_speed - correction;
24     int left_target = base_speed + correction;
25
26
27     if (left_target > max_speed) left_motor_pwm = max_speed;
28     else if (left_target < 0)    left_motor_pwm = 0;
29     else                        left_motor_pwm = left_target;
30
31     if (right_target > max_speed) right_motor_pwm = max_speed;
32     else if (right_target < 0)    right_motor_pwm = 0;
33     else                        right_motor_pwm = right_target;
34
35     __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, right_motor_pwm);
36     __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, left_motor_pwm);
37 }

```

Listing 3: Główna pętla sterowania P

```

1 void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
2 {
3     if (hadc->Instance == ADC1)
4     {
5         ADC_flag = 1;
6     }
7 }
8
9 int _write(int file, char *ptr, int len) {
10     HAL_UART_Transmit(&huart2, (uint8_t*)ptr, len, 50);
11     return len;
12 }

```

Listing 4: Przerwania i UART

6 Sterowanie PID

6.1 Kod programu

```

1
2 /*
3  * pid.h
4  *
5  * Created on: 09.03.2018
6  * Author: Wojciech Domski
7  */
8
9 #ifndef PID_H_
10 #define PID_H_
11

```

```

12 #include <stdint.h>
13
14 typedef struct {
15     int32_t p, i, d;
16
17     int32_t p_max, i_max, d_max;
18
19     int32_t p_min, i_min, d_min;
20
21     uint8_t f;
22     int32_t power;
23
24     int32_t sum;
25     int32_t e_last;
26
27     int32_t total_max;
28     int32_t total_min;
29
30     int32_t control;
31     int32_t dt_ms;
32 } cpid_t;
33
34 void pid_init(cpid_t * pid, float p, float i, float d, uint8_t f, int32_t
    dt_ms);
35
36 int32_t pid_calc(cpid_t * pid, int32_t mv, int32_t dv);
37
38 int32_t pid_scale(cpid_t * pid, float v);
39
40 #endif /* PID_H */

```

Listing 5: plik nagłówkowy pid.h

```

1 #include "pid.h"
2
3 void pid_init(cpid_t * pid, float p, float i, float d, uint8_t f, int32_t
    dt_ms) {
4     pid->f = f;
5     pid->power = (1 << f);
6
7     pid->p = (int32_t)(p * pid->power);
8     pid->i = (int32_t)(i * pid->power);
9     pid->d = (int32_t)(d * pid->power);
10
11     pid->p_max = INT32_MAX;
12     pid->p_min = INT32_MIN;
13
14     pid->i_max = INT32_MAX;
15     pid->i_min = INT32_MIN;
16
17     pid->d_max = INT32_MAX;
18     pid->d_min = INT32_MIN;
19
20     pid->dt_ms = dt_ms;
21
22     pid->sum = 0;
23     pid->e_last = 0;
24
25     pid->total_max = INT32_MAX;
26     pid->total_min = INT32_MIN;
27 }
28
29 int32_t pid_calc(cpid_t * pid, int32_t mv, int32_t dv) {
30     int32_t p_term, i_term, d_term, e, total;

```

```

31
32     e = dv - mv;
33
34     p_term = pid->p * e;
35
36     if (p_term > pid->p_max)
37         p_term = pid->p_max;
38     else if (p_term < pid->p_min)
39         p_term = pid->p_min;
40
41     i_term = pid->sum;
42
43     i_term += (pid->i * e * pid->dt_ms) / 1000;
44
45     if (i_term > pid->i_max)
46         i_term = pid->i_max;
47     else if (i_term < pid->i_min)
48         i_term = pid->i_min;
49
50     pid->sum = i_term;
51
52     d_term = (pid->d * (e - pid->e_last) * 1000) / pid->dt_ms;
53
54     if (d_term > pid->d_max)
55         d_term = pid->d_max;
56     else if (d_term < pid->d_min)
57         d_term = pid->d_min;
58
59     total = p_term + i_term + d_term;
60
61     if (total > pid->total_max)
62         total = pid->total_max;
63     else if (total < pid->total_min)
64         total = pid->total_min;
65
66     pid->control = total >> pid->f;
67     pid->e_last = e;
68
69     return pid->control;
70 }
71
72 int32_t pid_scale(cpid_t * pid, float v) {
73     return (int32_t)(v * (float)pid->power);
74 }

```

Listing 6: Plik źródłowy pid.c

```

1  uint16_t adc_results[8];
2  int ADC_flag;
3
4  cpid_t pid;
5
6  const int weights[8] = {40, 30, 20, 10, -10, -20, -30, -40};
7
8  int base_speed = 500;
9  int max_speed = 799;
10
11 int32_t motor_l = 0;
12 int32_t motor_r = 0;
13 int32_t u = 0;
14
15 float Kp = 15.0f;
16 float Ki = 0.1f;

```

```
17 float Kd = 1.0f;
```

Listing 7: zmienne globalne

```
1 // 1. Włączenie czujnika linii
2 HAL_GPIO_WritePin(LED_SENSORS_GPIO_Port, LED_SENSORS_Pin, GPIO_PIN_SET);
3 HAL_Delay(20);
4
5 // 2. Start pomiarów z czujnika
6 HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_results, 8);
7
8 // 3. Ustawienie kierunku silników
9 HAL_GPIO_WritePin(A_IN1_GPIO_Port, A_IN1_Pin, GPIO_PIN_RESET);
10 HAL_GPIO_WritePin(A_IN2_GPIO_Port, A_IN2_Pin, GPIO_PIN_SET);
11 HAL_GPIO_WritePin(B_IN1_GPIO_Port, B_IN1_Pin, GPIO_PIN_SET);
12 HAL_GPIO_WritePin(B_IN2_GPIO_Port, B_IN2_Pin, GPIO_PIN_RESET);
13
14 // 4. Włączenie silników
15 HAL_GPIO_WritePin(MD_STBY_GPIO_Port, MD_STBY_Pin, GPIO_PIN_SET);
16
17 // 5. PWM dla silników
18 __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 0);
19 __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 0);
20 HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
21 HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_2);
22
23 // 6. Naciśnięcie przycisku odpala robota
24 while (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_SET)
25 {
26     HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
27     HAL_Delay(100);
28 }
29
30 HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);
31 HAL_Delay(1000);
32
33 pid_init(&pid, Kp, Ki, Kd, 10, 1);
34
35 pid.total_max = pid_scale(&pid, 500.0f);
36 pid.total_min = pid_scale(&pid, -500.0f);
37
38 HAL_TIM_Base_Start(&htim6);
```

Listing 8: Inicjalizacja w main

```
1
2 // A – lewy
3 // B – prawy
4 // 1 – tył
5 // 2 – przód
6
7 while (1)
8 {
9     ADC_flag = 1;
10     if (ADC_flag == 1)
11     {
12         ADC_flag = 0;
13
14         int32_t numerator = 0;
15         int32_t denominator = 0;
16
17         for (int i = 0; i < 8; i++)
18         {
19             if (adc_results[i] > 500)
20             {
```



```

21         numerator += weights[i];
22         denominator++;
23     }
24 }
25
26 if (denominator == 8) {
27     motor_l = 0;
28     motor_r = 0;
29 }
30 else if (denominator > 0)
31 {
32     int32_t error = numerator / denominator;
33
34     u = pid_calc(&pid, error, 0);
35
36     motor_l = base_speed - u;
37     motor_r = base_speed + u;
38 }
39 else
40 {
41     if (pid.e_last > 0)
42     {
43         motor_l = 0;
44         motor_r = max_speed;
45     }
46     else if (pid.e_last < 0)
47     {
48         motor_l = max_speed;
49         motor_r = 0;
50     }
51 }
52
53 if (motor_l > max_speed) motor_l = max_speed;
54 if (motor_l < 0) motor_l = 0;
55 if (motor_r > max_speed) motor_r = max_speed;
56 if (motor_r < 0) motor_r = 0;
57
58 __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, (uint16_t)motor_r);
59 __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, (uint16_t)motor_l);
60 }
61 }

```

Listing 9: Główna pętla sterująca

```

1 void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
2 {
3     if (hadc->Instance == ADC1)
4     {
5         ADC_flag = 1;
6     }
7 }
8
9 int _write(int file, char *ptr, int len) {
10     HAL_UART_Transmit(&huart2, (uint8_t*)ptr, len, 50);
11     return len;
12 }

```

Listing 10: Przerwania i UART

6.2 Opis pętli sterującej

Algorytm PID został zaimplementowany w oparciu o [4] Sterowanie robotem realizowane jest w pętli głównej mikrokontrolera. Algorytm opiera się na cyklicznym odczycie wartości z listwy czujników (transoptorów odbiciowych), wyznaczeniu uchybu regulacji oraz wysterowaniu silników DC za

pomocą sygnału PWM. Cały proces można podzielić na trzy etapy: akwizycję danych, obliczenia regulatora PID oraz obsługę sytuacji wyjątkowych (zgubienie linii).

6.2.1 Odczyt danych i wyznaczenie uchybu

Pozycja linii względem środka robota wyznaczana jest na podstawie odczytów z 8-kanalowej listwy czujników. Każdy czujnik zwraca wartość analogową, która jest poddawana binaryzacji z progiem 500 (wartość umowna z przetwornika ADC).

Uchyb regulacji $e(k)$ w chwili k obliczany jest jako średnia ważona aktywnych czujników, co pozwala na precyzyjne określenie położenia linii nawet przy niskiej rozdzielczości listwy.

Jeśli linia znajduje się w polu widzenia czujników, sterowanie przejmuje regulator PID. Sygnał sterujący $u(k)$ obliczany jest według wzoru:

$$u(k) = K_p \cdot e(k) + K_i \cdot \sum_{j=0}^k e(j) + K_d \cdot [e(k) - e(k-1)] \quad (1)$$

gdzie K_p, K_i, K_d to odpowiednio wzmocnienia członu proporcjonalnego, całkującego i różniczkującego.

Prędkości zadane dla silnika lewego (V_L) i prawego (V_R) wyznaczane są poprzez sumowanie prędkości bazowej (V_{base}) z wyjściem regulatora:

$$\begin{cases} V_L = V_{base} - u(k) \\ V_R = V_{base} + u(k) \end{cases} \quad (2)$$

Wartości te są następnie ograniczone do zakresu dopuszczalnego sterowania PWM $\langle 0, V_{max} \rangle$.

6.2.2 Algorytm wykrywania zgubienia linii i pokonywania ostrych zakrętów

W przypadku, gdy żaden z czujników nie wykrywa linii, co zdarza się przy ostrych zakrętach (np. 90°) lub zygzakach, algorytm przechodzi w specjalny tryb.

Wykorzystywana jest wówczas zmienna e_{last} (ostatni znany uchyb), która przechowuje informację, po której stronie robota linia była widoczna jako ostatnia. Pozwala to na podjęcie decyzji o gwałtownym skręcie w kierunku zakładanej pozycji trasy:

- Jeżeli $e_{last} > 0$ (linia uciekła w prawo): Silnik lewy zostaje zatrzymany ($V_L = 0$), a prawy wystawiony na maksymalną prędkość ($V_R = V_{max}$), co powoduje ostry skręt w lewo.
- Jeżeli $e_{last} < 0$ (linia uciekła w lewo): Silnik prawy zostaje zatrzymany ($V_R = 0$), a lewy wystawiony na maksymalną prędkość ($V_L = V_{max}$), co powoduje ostry skręt w prawo.

Mechanizm ten pozwala robotowi na powrót na trasę nawet w przypadku utraty ciągłości sygnału sterującego z pętli PID.

6.3 Dobór nastaw regulatora PID

Dobór parametrów regulatora przeprowadzono metodą eksperymentalną, bazując na obserwacji zachowania robota na torze testowym. Celem strojenia było uzyskanie kompromisu pomiędzy szybkością reakcji na zakrętach a stabilnością jazdy na odcinkach prostych. Ostatecznie przyjęto następujące wartości nastaw:

- **Wzmocnienie proporcjonalne** ($K_p = 15.0$): Parametr ten stanowił główny czynnik sterujący. Wartość 15.0 została dobrana tak, aby zapewnić wystarczający moment skręcający silników przy wejściu w ostre łuki. Niższe wartości powodowały wypadanie robota z trasy na zakrętach, natomiast wyższe prowadziły do silnych oscylacji wokół linii.

- **Wzmocnienie różniczkujące ($K_d = 10.0$):** Wysoka wartość członu różniczkującego okazała się kluczowa dla stabilizacji robota przy tak dużym wzmocnieniu K_p . Człon D pełni rolę „tłumika”, przeciwdziała gwałtownym zmianom uchybu, redukując przeregulowania po wyjściu z zakrętu. Wartość 10.0 pozwoliła na szybkie wygaszanie oscylacji.
- **Wzmocnienie całkujące ($K_i = 0.1$):** Ze względu na dynamiczny charakter robota (Line Follower), człon całkujący został ograniczony do minimum. Niewielka wartość 0.1 służy do dociągnięcia robota do linii na zakrętach, ale nie powoduje nadmiernych oscylacji na liniach prostych.

Zestawienie przyjętych nastaw prezentuje Tabela 8.

Tabela 8: Ostateczne nastawy regulatora PID

Parametr	Wartość	Funkcja w układzie
K_p	15.0	Szybkość reakcji i siła skrętu
K_i	0.1	Eliminacja uchybu i dociągnięcie robota
K_d	10.0	Tłumienie oscylacji i predykcja

7 Trasa robota

Trasa robota została wykonana z 8 kartek A3 oraz taśmy klejącej o szerokości 20mm, co pozwala łapać linie 2 lub 3 czujnikom podczas jazdy robota do przodu. Zawiera ona bardzo ostre skrzyty, zygzaki, zakręty 90 stopni, długą prostą oraz skręty o maksymalnym promieniu, gdzie robot nie wypadnie poza trasę.



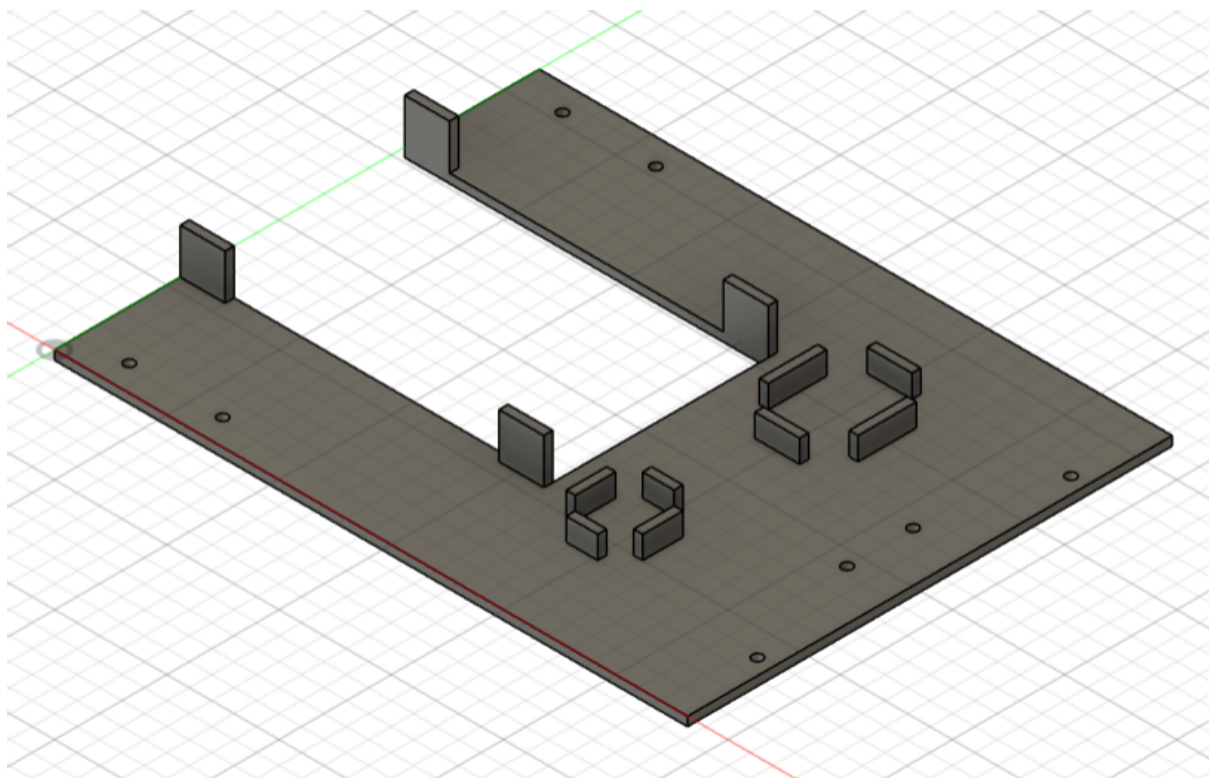
Rysunek 5: Trasa robota

8 Konstrukcja mechaniczna

Realizacja projektu wymagała skonstruowania podwozia, czyli podstawy robota, do którego przy-mocowano pozostałe części. Podwozie jako pozostałe elementy, takie jak pojemnik na koszyk z bateriami, mocowania dla silników, mocowania dla listwy z czujnikami oraz podkładkę pod koło podporowe zaprojektowano w programie Autodesk Fusion. Zaprojektowane elementy zostały na-stępnie wydrukowane w druku 3D.

8.0.1 Podwozie robota

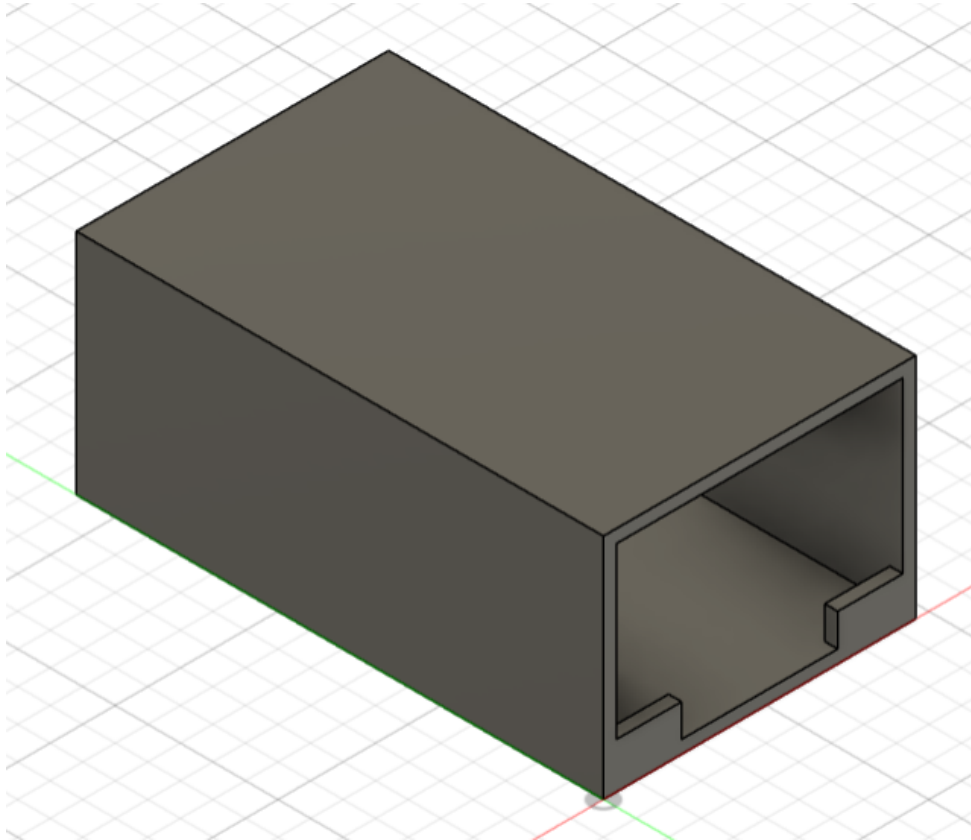
Podwozie robota zostało zaprojektowane z myślą o praktycznym rozmieszczeniu na nim wszystkich części mechanicznych jak i urządzeń zewnętrznych.



Rysunek 6: Projekt podwozia robota

8.1 Pojemnik na koszyk z bateriami

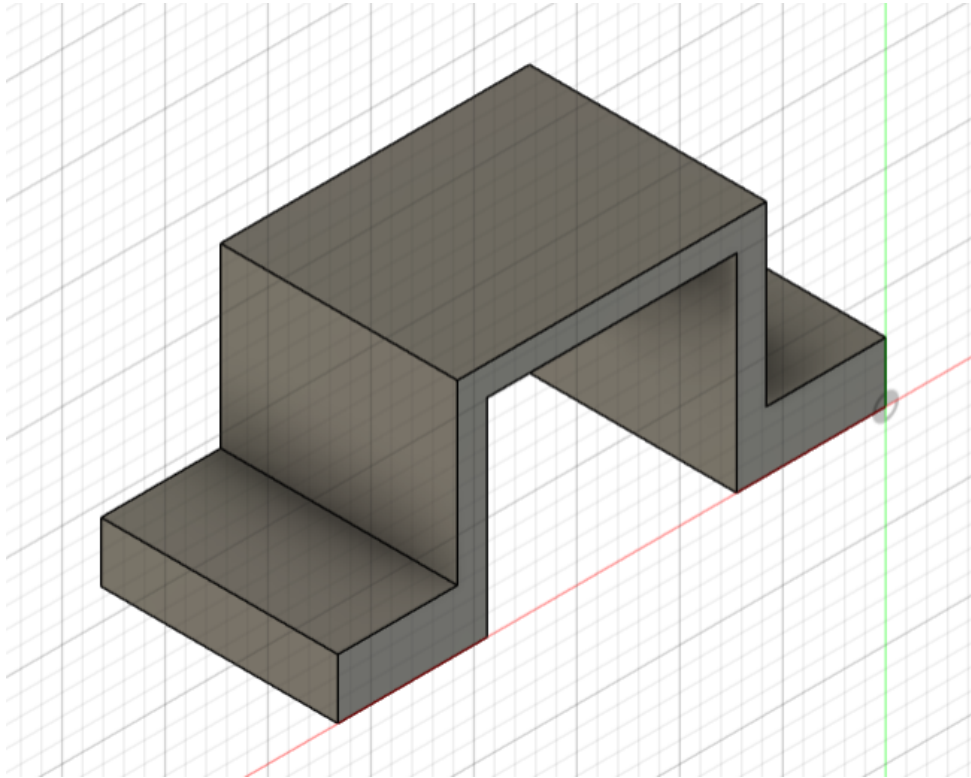
Pojemnik na koszyk z bateriami został stworzony z myślą o wykorzystaniu wolnej przestrzeni znajdującej się między podstawą a podłogą w tylnej części robota. Ponadto jego górna powierzchnia zapewnia idealną przestrzeń na umieszczenie mikrokontrolera.



Rysunek 7: Projekt pojemnika na koszyk z bateriami

8.2 Mocowania silników

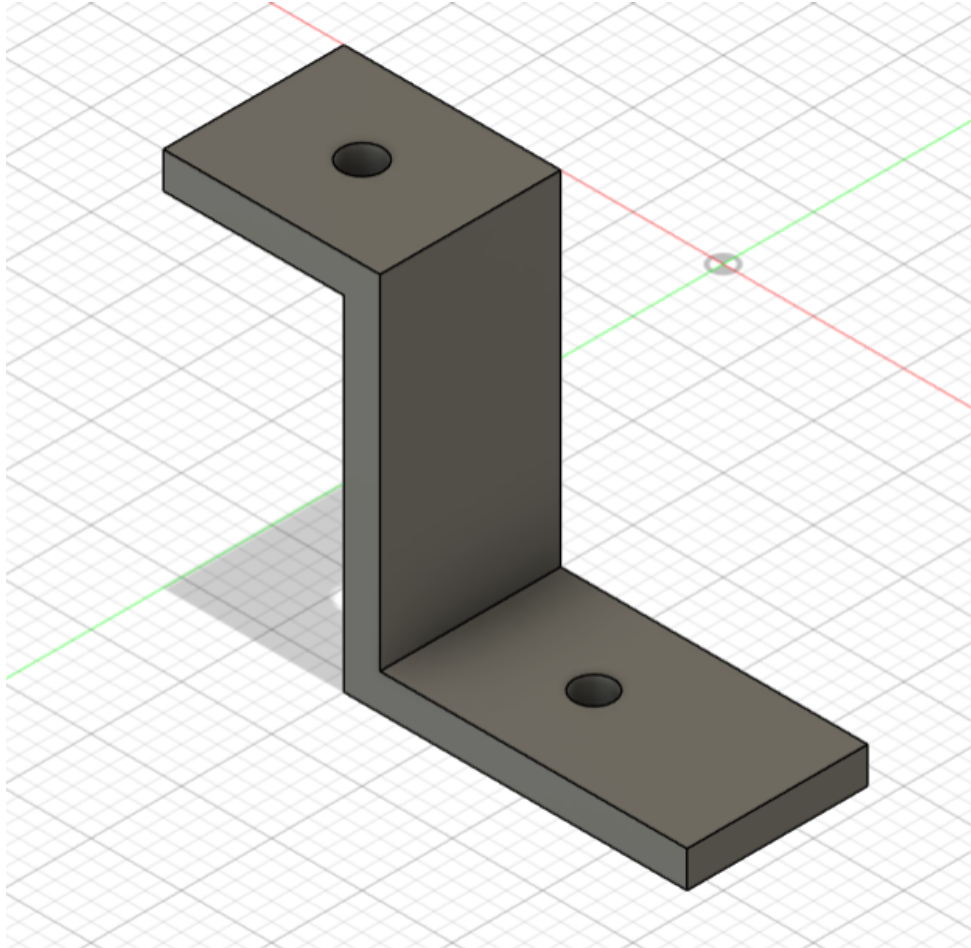
Mocowania silników zostały zaprojektowane w celu utrzymywania ich w jednej pozycji. Ich zadanie mimo tego, że jest bardzo proste okazuje się być niezwykle istotne, gdyż poprawna pozycja silników jest niezwykle kluczowa dla poprawnego przemieszczania się robota.



Rysunek 8: Projekt pojedynczego mocowania silnika

8.3 Mocowania listwy z czujnikami

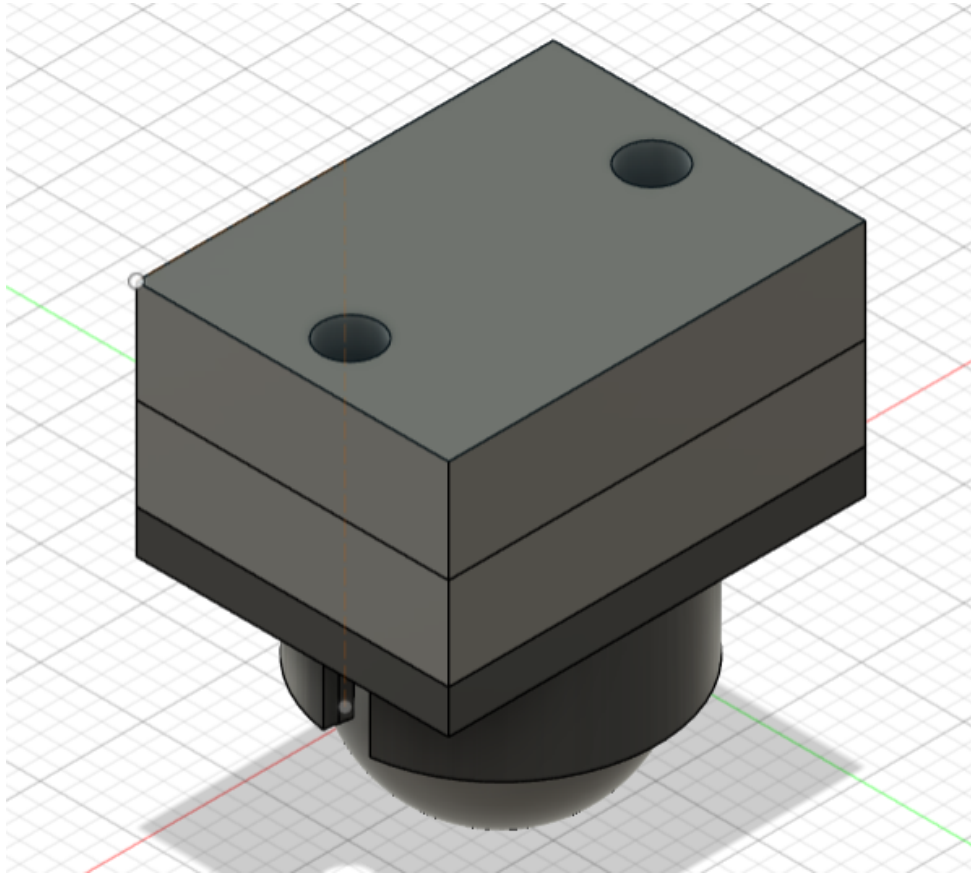
Mocowania dla listwy z czujnikami zostały zaprojektowane w celu utrzymywania jej w jednej pozycji. Ich zadanie mimo tego, że jest bardzo proste okazuje się być niezwykle istotne, gdyż poprawna pozycja listwy jest niezwykle kluczowa dla poprawnego działania czujników znajdujących się na niej oraz tym samym śledzenia linii, po której poruszać ma się robot.



Rysunek 9: Projekt pojedynczego mocowania listwy z czujnikami

8.4 Podkładka po koło podporowe

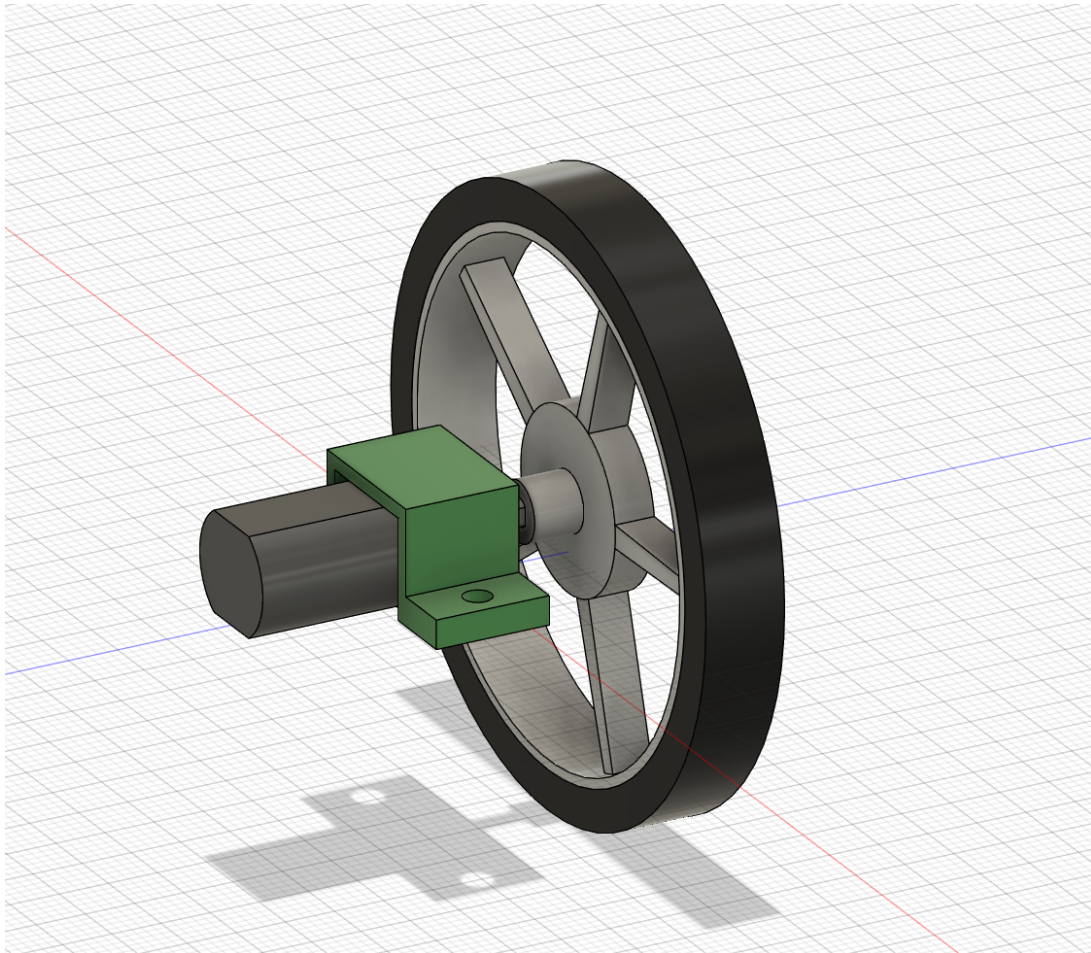
Podkładka pod koło podporowe została zaprojektowana w celu wyrównania podstawy robota względem podłogi. Wykorzystanie jej zapewnia równoległą pozycję podwozia oraz listwy z czujnikami względem powierzchni po której przemieszcza się robot.



Rysunek 10: Projekt podkładki pod koło podporowe

8.5 Moduł napędowy

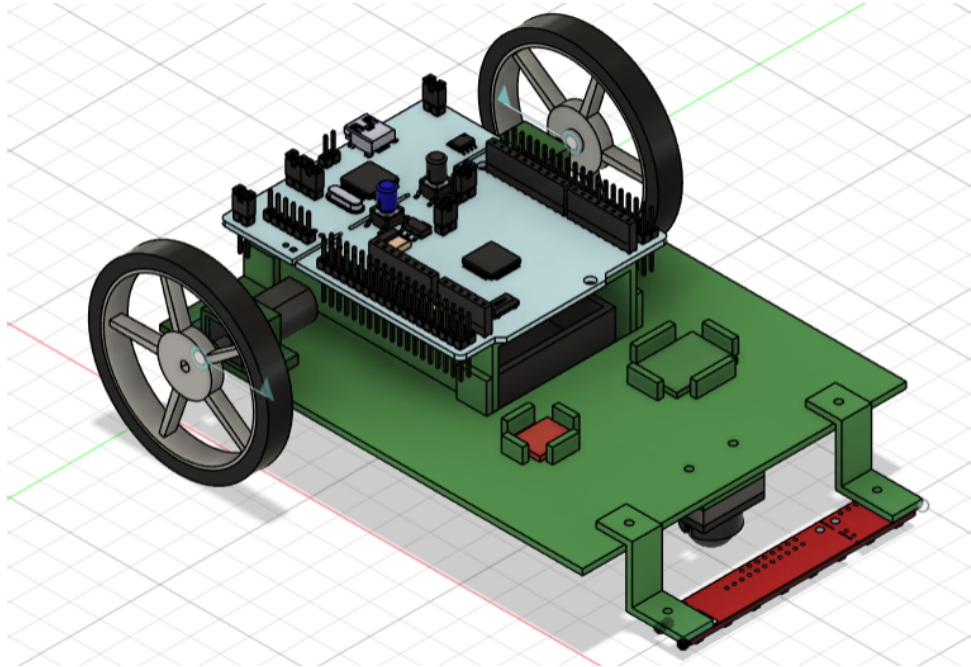
W ramach ćwiczenia programu Fusion samodzielnie zaprojektowano dodatkowe elementy składające się w moduł napędowy takie jak koło, silnik i mocowanie. Wykonano je jako osobne bryły, które następnie złożono w jeden spójny moduł napędowy. Dodano również system animacji przedstawiający obrót koła.



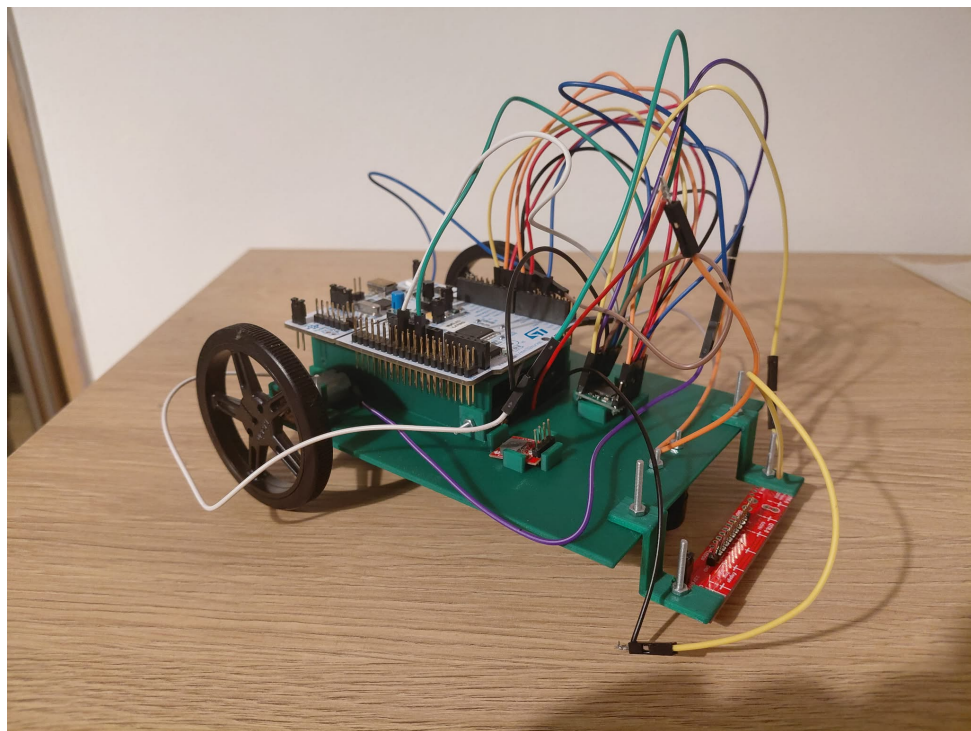
Rysunek 11: Moduł napędowy

8.6 Złożenie robota

Implementacja dodatkowych elementów konstrukcji mechanicznej wraz z pozostałymi elementami niezbędnymi do zrealizowania projektu pozwalają na zasymulowanie tego, jak robot powinien wyglądać w rzeczywistości.

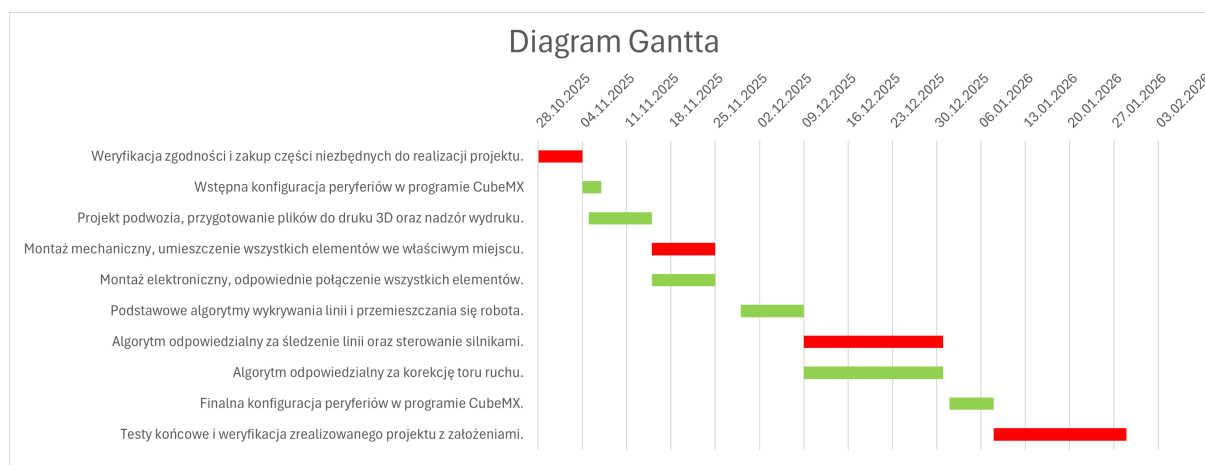


Rysunek 12: Złożenie robota - symulacja



Rysunek 13: Złożenie robota - rzeczywistość

9 Harmonogram pracy



Rysunek 14: Diagram Gantta

Na powyższym diagramie Gantta kolorem czerwonym oznaczone zostały kamienie milowe oraz ścieżka krytyczna. Na diagramie tym można wyróżnić następujące kamienie milowe:

- Weryfikacja, zatwierdzenie i zamówienie komponentów
Odpowiedni dobór elementów niezbędnych do realizacji projektu jest wyjątkowo istotny. Wszystkie elementy muszą zostać prawidłowo dobrane, aby cel projektu mógł zostać osiągnięty.
- Konstrukcja mechaniczna robota
Odpowiednie połączenie elementów mechanicznych w jedną całość. Kluczowe jest właściwe rozmieszczenie elementów oraz zamocowanie komponentów. Fizyczne złożenie robota jest niezbędne, aby móc przejść do kolejnych etapów.
- Algorytm śledzenia linii oraz sterowania silnikami
Jest to najważniejszy algorytm, ponieważ odpowiada bezpośrednio za realizację głównego zadania robota, czyli podążanie za wyznaczoną linią. Jest kluczowy do kompletnej realizacji projektu.
- Testy końcowe, finalizacja projektu.
Ostateczna weryfikacja całego systemu, wykonanej pracy i potwierdzenie zgodności z założeniami.

9.1 Zakres prac

- Weryfikacja zgodności i zakup części niezbędnych do realizacji projektu.
- Wstępna konfiguracja peryferiów w programie CubeMx.
- Projekt podwozia, przygotowanie plików do druku 3D oraz nadzór wydruku.
- Montaż mechaniczny, umieszczenie wszystkich elementów we właściwym miejscu.
- Montaż elektroniczny, odpowiednie połączenie wszystkich elementów.
- Podstawowy algorytm wykrywania linii.
- Podstawowy algorytm przemieszczania się robota.
- Algorytm odpowiedzialny za śledzenie linii oraz sterowanie silnikami.
- Algorytm odpowiedzialny za korekcję toru ruchu.
- Finalna konfiguracja peryferiów w programie CubeMX.
- Testy końcowe i weryfikacja zrealizowanego projektu z założeniami.

9.2 Podział pracy

Mateusz Duda	%	Łukasz Lysek	%
Weryfikacja zgodności i zakup części niezbędnych do realizacji projektu	100	Weryfikacja zgodności i zakup części niezbędnych do realizacji projektu	100
Projekt podwozia, przygotowanie plików do druku 3D oraz nadzór wydruku	100	Wstępna konfiguracja peryferiów w programie CubeMx	100
Montaż elektroniczny, odpowiednie połączenie wszystkich elementów	90	Montaż mechaniczny, umieszczenie wszystkich elementów we właściwym miejscu	80
Podstawowy algorytm wykrywania linii	75	Podstawowy algorytm przemieszczania się robota	75

Tabela 9: Podział pracy – Etap II

Mateusz Duda	%	Łukasz Lysek	%
Algorytm odpowiedzialny za śledzenie linii oraz sterowanie silnikami		Algorytm odpowiedzialny za korekcję toru ruchu	
		Finalna konfiguracja peryferiów w programie CubeMX	
Testy końcowe i weryfikacja zrealizowanego projektu z założeniami		Testy końcowe i weryfikacja zrealizowanego projektu z założeniami	

Tabela 10: Podział pracy – Etap III

10 Podsumowanie

Projekt robota podążającego za linią to ciekawy i praktyczny przykład robota, który potrafi samodzielnie poruszać się po wyznaczonej linii. Wymaga podstawowych umiejętności z zakresu programowania mikrokontrolerów, sterowania silnikami, odczytu danych z czujników oraz tworzenia konstrukcji mechanicznych. Projekt ten w bardzo dobry sposób pokazuje, jak przy pomocy stosunkowo niewielu elementów można stworzyć robota realizującego określone zadanie.

Literatura

- [1] W. Domski. Sterowniki robotów, Laboratorium – ADC, DAC i DMA, Przetwoniki analogowo–cyfrowe, cyfrowo–analogowe oraz bezpośredni dostęp do pamięci. Mar. 2017.
- [2] W. Domski. Sterowniki robotów, Laboratorium – Debugowanie, Zaawansowane techniki debugowania. Mar. 2017.
- [3] W. Domski. Sterowniki robotów, Laboratorium – Liczniki i przerwania, Tryby pracy licznika oraz obsługa przerwań. Mar. 2017.
- [4] W. Domski. Sterowniki robotów, Laboratorium – Regulator PID, Implementacja i strojenie regulatora PID na bazie symulatora silnika. Mar. 2017.
- [5] M. Salamon. ADC w STM32 na kilka sposobów. 2019.